

Minho University
Engineering School

Dong Xuyong

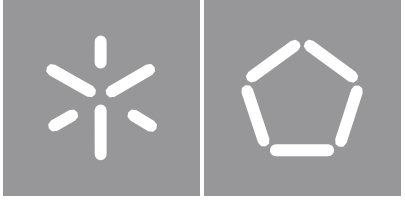
Hierarchical structure optimization for anomaly detection in grouped time series data

Hierarchical structure optimization
for anomaly detection in grouped time series data

Dong Xuyong

UMinho | 2024

February 2025



Minho University
Engineering School

Dong Xuyong

Hierarchical structure optimization for anomaly detection in grouped time series data

Master's dissertation in Information Systems Engineering and Management

Work carried out under the supervision of
Professor Doctor Paulo Alexandre Ribeiro Cortez
Professor Doctor Emanuel Demétrio Mendes Gouveia

February 2025

Acknowledgements

Summary

Esta dissertação foca-se na deteção de anomalias em séries temporais de telecomunicações grouped time series provenientes de uma rede real. Propõe um método agnóstico relativamente ao detetor para descobrir estruturas de agregação hierárquica e aplicá-las à deteção de anomalias em dados agrupados complexos. Em vez de depender apenas de uma agregação global única ou de uma monitorização totalmente desagregada, o trabalho explora estruturas hierárquicas totais ou parciais que podem revelar padrões anómalos de tráfego a um nível de detalhe útil.

A investigação analisa como a procura evolutiva pode apoiar uma deteção de anomalias mais flexível e interpretável em dados de monitorização de rede com atributos categóricos hierárquicos. As hierarquias candidatas são geradas e avaliadas pela capacidade das séries agregadas resultantes de separar comportamento normal de anómalo. A abordagem combina agregação a várias escalas, injeção de anomalias sintéticas, pontuação baseada em Matrix Profile e Grammatical Evolution com PonyGE2.

Abstract

This dissertation focuses on anomaly detection in grouped telecommunications time-series data from a real network. Modern telecommunications networks generate large volumes of traffic that must be monitored to detect anomalous behaviour that can affect service reliability, capacity provisioning, and operational decision-making. These datasets are structured along several categorical dimensions, which creates many possible aggregation levels and makes it difficult to decide which time series to analyse. This work proposes a detector-agnostic framework based on hierarchical aggregation structures. A single global view or fully disaggregated monitoring is often inadequate: a global view can hide local anomalies, while full disaggregation is costly and can leave many series too sparse for reliable detection. Instead, this work explores partial hierarchical structures that may reveal abnormal traffic patterns at a relevant level of detail, aiming to detect anomalies in complex grouped datasets without sacrificing interpretability or incurring prohibitive analytical cost.

A formal grammar automatically generates hierarchical structures, which are evaluated by their ability to distinguish normal from anomalous behaviour. In addition to predefined hierarchies, the method can discover useful aggregation structures automatically. To this end, Grammatical Evolution (*Grammatical Evolution* (GE)) (O'Neill and Conor Ryan, 2001; O'Neill and C. Ryan, 2003) is used. The main contribution is a reproducible, interpretable framework for learning hierarchical anomaly-detection rules for telecommunications time series, combining evolutionary search with time-series anomaly scoring despite the dimensionality of the network data.

Keywords: anomaly detection, telecommunications, hierarchical time series, grouped time series, Grammatical Evolution, Matrix Profile.

Contents

Acknowledgements . . . 2
Summary . . . 3
Abstract . . . 4
List of Figures . . . 7
List of Tables . . . 8
List of abbreviations and acronyms . . . 9

1 Introduction **10**
1.1 Motivation . . . 10
1.2 Objectives and Expected Results . . . 10
1.3 Document Organization . . . 11
1.4 Search Method . . . 11

2 Theory and Background **12**
2.1 Time Series . . . 12
2.1.1 Hierarchical Time Series . . . 12
2.1.2 Grouped Time Series . . . 13
2.2 Anomaly Detection . . . 14
2.3 Anomaly Generation . . . 14
2.3.1 Rule-Based Approaches . . . 15
2.3.2 Validation of Synthetic Anomalies . . . 15
2.4 Grammatical Evolution . . . 15
2.4.1 Combinatorial Optimization . . . 15
2.4.2 Genetic Programming . . . 15
2.4.3 Backus-Naur Form . . . 16
2.4.4 Initialization Phase . . . 17
2.4.5 Selection Mechanism . . . 18
2.4.6 Variation Operators . . . 18
2.4.7 Replacement Strategy . . . 18
2.5 Metrics . . . 18
2.5.1 Precision and Recall . . . 18
2.5.2 Trade-off and Applications . . . 19
2.5.3 Area Under the ROC Curve (AUC) . . . 19

3 Literature Review **21**
3.1 Anomaly Detection in Telecommunications Networks . . . 21
3.2 Anomaly Detection in Grouped Time Series . . . 22
3.3 Anomaly Detection in Hierarchical Time Series . . . 22
3.4 Matrix Profile and Time-Series Discord Detection . . . 23

3.5 Combinatorial Optimisation for Selecting Time Series 23

4 Materials and Methodology **25**
4.1 Problem Formulation 25
4.1.1 Key Observations 26
4.1.2 Analytical Implications 27
4.2 Adopted Methodology 27
4.3 Model Development and Evaluation 30
4.3.1 Genome Structure 30
4.4 Anomaly Injection 31
4.5 Hierarchical Data Structures 31
4.6 Genetic Programming Implementation 32

5 Results and Discussion **33**
5.1 Performance Analysis 33
5.2 Time Series Visualization 33
5.3 Genetic Programming Analysis 33

6 Conclusions and Future Works **34**
6.1 Conclusions 34
6.2 Future Work 34

List of Figures

2.1 A two-level hierarchical tree diagram 13
2.2 Australian domestic overnight trips, 1998 Q1–2017 Q4, by purpose and state 14
2.3 Generational loop in an evolutionary algorithm 16

4.1 Vodafone network traffic metrics (#Events and Event Duration) over July-October 2022 aggregated. 25
4.2 Daily #Events from 18–28 July 2022 26
4.3 Anomaly detection workflow 28
4.4 PonyGE2 control flow diagram. 30
4.5 Anomaly injection cycle 31

List of Tables

4.1 Cardinality and Dominant Categories in Vodafone Data 26
4.2 Information Content by Variable 27
4.3 Key Configuration Parameters 32

List of abbreviations and acronyms

AUC *Area Under the ROC Curve*

BNF *Backus–Naur Form*

CDR *Call Detail Record*

CNN *Convolutional Neural Network*

GA *Genetic Algorithm*

GE *Grammatical Evolution*

GP *Genetic Programming*

ISP *Internet Service Provider*

KPI *Key Performance Indicator*

LSTM *Long Short-Term Memory*

MAE *Mean Absolute Error*

MP *Matrix Profile*

PIG *Position-Independent Grow*

ROC *Receiver Operating Characteristic*

ROI *Region of Interest*

VAE *Variational Autoencoder*

Chapter 1

Introduction

1.1 Motivation

Unexpected changes in ordinary traffic patterns can occur at any instant and may result in network faults or quality-of-service problems. Telecom traffic is not a single univariate series; it can be viewed under several categorical dimensions. For example, a call can have different types, an event type may take many values, and a subscriber may belong to a given type. Traffic type, service type, partner-related information, and related categories create many candidate aggregations on which anomaly detection can be run. Expert labels for real incidents are scarce, so candidate aggregations cannot be ranked with standard supervised metrics. This dissertation focuses on injecting synthetic anomalies into each candidate aggregation and using *Area Under the ROC Curve* (AUC) on Matrix Profile discord scores as a label-free fitness signal.

Having a fully aggregated view can hide local anomalies, while monitoring every categorical combination separately is costly computationally and in terms of interpretability. Moreover, some fully disaggregated series contain so few events that anomalies cannot be identified with confidence. This creates a dilemma: which aggregation level should anomaly detection work on? Operators may use a fixed hierarchy or manual rules; for example, aggregating by protocol or subscriber type can make some anomalies visible. Running anomaly detection independently on every variable combination, however, remains expensive and hard to interpret. Therefore, automatic search methods are needed to identify suitable candidate aggregations.

1.2 Objectives and Expected Results

This dissertation aims to develop a framework for anomaly detection in grouped telecommunications time series with hierarchical aggregation, using Grammatical Evolution to evolve programmes from a formal grammar. From a data perspective, the aim is to work with the categorical dimensions of real mobile-network traffic records—call type, event type, subscriber type, traffic type, service type, partner-related information, and related fields—so that anomalies can be studied at global and local aggregation levels.

From a methodological point of view, aggregation choice is framed as a combinatorial search over those categorical dimensions. Candidate hierarchies are encoded in a *Backus–Naur Form* (BNF) grammar so that only valid aggregation structures are generated. Grammatical Evolution (PonyGE2) then evolves aggregation programmes from that

grammar. For each candidate, an aggregated series is built and scored with Matrix Profile (STUMPY) discord distances in a label-free way. Because anomaly labels are scarce, synthetic anomalies provide the evaluation signal: AUC on the discord scores is used as fitness, and evolved or partial hierarchies are compared with fixed aggregation baselines. The implementation uses real telecommunications network records, with preprocessing, aggregation, anomaly injection, the BNF grammar, and PonyGE2.

1.3 Document Organization

The dissertation is organised into six chapters. Chapter 1 presents the motivation, objectives, and document organisation. Chapter 2 reviews the theoretical background: time series, hierarchical and grouped time series, anomaly detection, anomaly generation, Grammatical Evolution, and evaluation metrics. Chapter 3 examines the literature on anomaly detection in telecommunication networks, grouped and hierarchical time series, Matrix Profile-based anomaly detection, combinatorial optimisation for selecting time series or structures, and Grammatical Evolution as an optimisation method. Chapter 4 describes the telecommunications dataset, preprocessing and aggregation, synthetic anomaly injection, hierarchy construction, and the PonyGE2 implementation. Chapter 5 presents the experimental results, comparing aggregation strategies and examining the evolved hierarchical structures. Chapter 6 summarises the contributions and limitations of the present work, and outlines future work, including the use of real incident labels and online detection.

1.4 Search Method

To support this dissertation, a layered bibliographic search was used. The first step built a synonym and subtopic list covering grouped and hierarchical time series, telecom anomaly detection, Matrix Profile, combinatorial optimisation, Grammatical Evolution, and anomaly injection, and ran Boolean queries in Google Scholar and IEEE Xplore, with additional checks on SpringerLink and other publisher sites as needed. The queries focused on keywords such as “Multivariate Time Series”, “Grouped time series”, “Hierarchical Time Series Forecasting”, “Hierarchical anomaly detection”, “Time Series and Anomaly Detection in Telecommunication”, “Combinatorial Optimization”, “Anomaly injection”, and “Matrix Profile”.

Because the combined topic is sparse and hard to retrieve by keywords alone, LLM-based systems were also used to surface additional candidate papers and related terms. From a small set of highly relevant seed papers, “Cited by” and related-work links were followed to catch papers the synonym list missed.

Candidate articles were screened primarily by relevance to the research gap (telecom anomaly detection, aggregation and hierarchy, Matrix Profile and discord detection, combinatorial or evolutionary search, and Grammatical Evolution). Publication date (preferring work after 2015 where appropriate), venue quality, and citation impact were used as secondary signals, not as a fixed citation cut-off. Core databases for this stage were IEEE Xplore, Google Scholar, SpringerLink, MDPI, and PLOS.

Chapter 2

Theory and Background

2.1 Time Series

A time series is a sequence of observations indexed in time order, typically collected at regular intervals (e.g., hourly or daily) (Box et al., 2015). Irregularly spaced data (e.g., event-driven recordings) can also be treated as time-series data in a broader sense. Methodological rigor requires consistency in measurement definitions so that observations remain comparable over time. Time-series data are univariate if they contain a single variable; multivariate series incorporate additional variables while preserving temporal alignment and relevance.

2.1.1 Hierarchical Time Series

Hierarchical time series are structured time-series datasets arranged into hierarchical relationships. Typical hierarchies include geographical areas (e.g., national → state → city) or product categories (e.g., product group → SKU, stock keeping unit).

Hierarchical time series exhibit multi-level aggregation, where lower-level components combine through arithmetic operations (e.g., summation) to reconstruct higher-level series (R. J. Hyndman and G. Athanasopoulos, 2021). For example, consider a geographic hierarchy:

- **Level 1 (National)**: Total sales in a country.
- **Level 2 (Regional)**: Sales in the *North* and *South* regions.
- **Level 3 (Local)**: Sales in cities (e.g., *North: City A + City B*; *South: City C + City D*).

The national series equals the sum of the regional series (National = North + South), and each regional series equals the sum of its constituent cities. This coherence ensures consistency across levels (Wickramasuriya et al., 2019).

When analysing hierarchical time series, concurrent analysis at several aggregation levels can balance computational efficiency with granularity (R. J. Hyndman and G. Athanasopoulos, 2018).

As shown in Figure 2.1, hierarchical time series arrange temporal data into several aggregation levels. The framework consists of:

- **Root level**: the whole temporal sequence is represented by the aggregate series y_t for $t = 1, \ldots, T$;

- **Intermediate levels**: disaggregated elements (e.g., $y_{A,t}$ and $y_{B,t}$) that partition the root series;
- **Leaf level**: the most detailed constituents (e.g., $y_{AA,t}$, $y_{AB,t}$, $y_{AC,t}$ under node A; $y_{BA,t}$, $y_{BB,t}$ under node B).

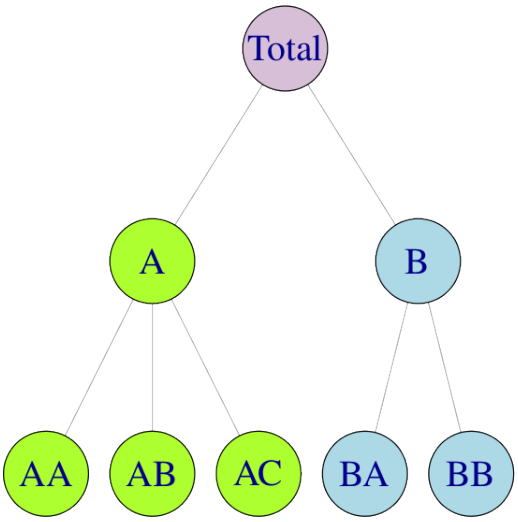


Figure 2.1: A two-level hierarchical tree diagram (R. J. Hyndman and G. Athanasopoulos, 2018).

Approaches to Hierarchical Forecasting

Forecasting within a hierarchical system uses techniques to ensure that forecasts are credible and internally coherent: the aggregation of forecasts at one level should match the forecast of the level above. The main approaches are bottom-up, top-down, and middle-out (R. J. Hyndman and G. Athanasopoulos, 2018).

The bottom-up approach begins its predictions at the base level and aggregates them to higher levels. This approach is computationally practical and often performs well when upper-level series are relatively stable. The top-down approach starts at the top of the hierarchy and distributes forecasted totals using historical proportions. The middle-out approach connects bottom-up and top-down methods by focusing on a mid-level, aiming to balance the strengths of both.

2.1.2 Grouped Time Series

Grouped time series represent collections of temporally structured data organised by shared attributes (e.g., device type, customer demographics) rather than by a single nested hierarchy. Unlike hierarchical series, grouped data lack inherent aggregation constraints: forecasts for individual series may be developed independently and then analysed collectively for cross-sectional patterns (R. J. Hyndman and G. Athanasopoulos, 2018). Grouped time series are widely used in macroeconomic analysis (George Athanasopoulos et al., 2017) and supply-chain forecasting (Wickramasuriya et al., 2015), where multiple cross-sectional dimensions demand coherent alignment without a strict hierarchical dependence.

Often, disaggregating factors are both nested and crossed. For example, Australian tourism data can be disaggregated by hierarchical geographical categories (states, and regions nested within states), but also by four purposes of travel: holiday, business, visiting friends and relatives, and other. This grouping variable does not nest within any of the geographical variables.

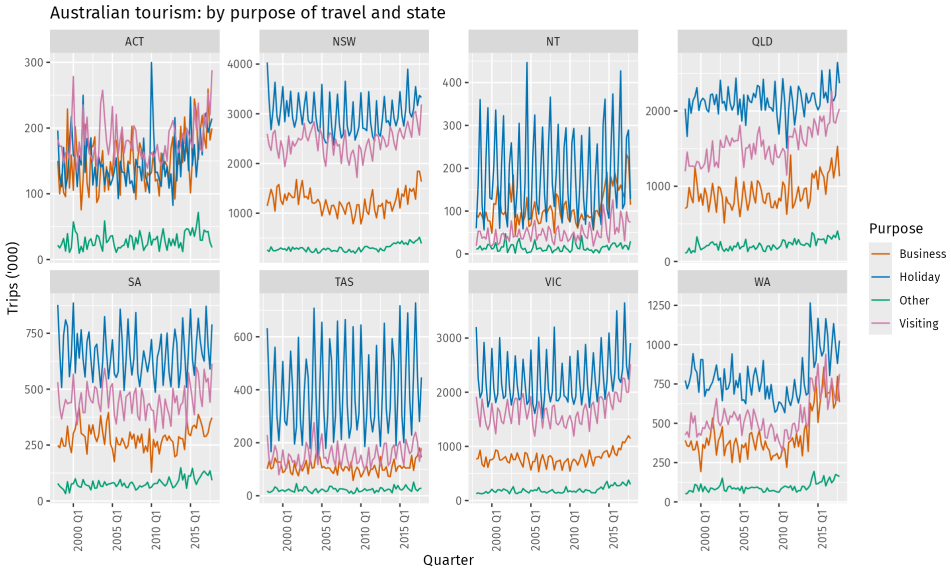


Figure 2.2: Australian domestic overnight trips over the period 1998 Q1 to 2017 Q4 disaggregated by purpose of travel and by state (R. J. Hyndman and G. Athanasopoulos, 2021).

2.2 Anomaly Detection

In machine learning, anomaly detection aims to find patterns that deviate significantly from expected behaviour (Chandola et al., 2009). It is essential in many fields, including fault diagnosis, network security, and fraud detection. Univariate anomaly detection identifies outliers in a single variable. Multivariate anomaly detection considers several variables jointly, so that a point may be anomalous because of an unusual combination of values even when each variable looks normal in isolation.

Anomalies are often classified as point anomalies (a single observation is unusual), collective anomalies (a contiguous subsequence is unusual as a group), or contextual anomalies (an observation is unusual only in a given context) (Chandola et al., 2009). These distinctions matter later for Matrix Profile scoring, which is strongest for unusual short subsequences (discords) and weaker for some sustained level shifts or purely relational multivariate faults.

2.3 Anomaly Generation

Anomaly generation is the methodical production of irregular patterns or outliers that depart significantly from normal data distributions (Hawkins, 1980). In machine learning settings it is used both to stress-test system robustness (Chandola et al., 2009) and to

augment training or evaluation data. Modern practice emphasises generating semantically plausible contextual anomalies for the target domain (Ruff et al., 2021).

2.3.1 Rule-Based Approaches

Expert-defined constraints in domain-specific anomaly generation deliberately break normal system behaviour (Chandola et al., 2009). In this dissertation, synthetic anomalies are injected with rule-based thresholds (e.g., values outside $\mu \pm 3\sigma$ over a 168-hour window).

2.3.2 Validation of Synthetic Anomalies

To ensure synthetic anomalies are useful for evaluation, validation typically combines domain checks (e.g., expert judgement that injected patterns are realistic) with quantitative detection benchmarks (Ruff et al., 2021). In this work, injected labels enable AUC evaluation of Matrix Profile discord scores on each candidate aggregation.

2.4 Grammatical Evolution

2.4.1 Combinatorial Optimization

Combinatorial optimisation identifies optimal configurations within finite discrete solution spaces and provides formal frameworks for many NP-hard problems (C. H. Papadimitriou and K. Steiglitz, 1982; Garey and Johnson, 1979). Exact methods (e.g., integer programming, branch-and-bound, dynamic programming) guarantee optimality but can be computationally costly at scale (Christos H Papadimitriou and Kenneth Steiglitz, 1998). Approximation and metaheuristic methods—including greedy search, local search, simulated annealing, and genetic algorithms—trade guarantees for tractable near-optimal solutions on large instances (Vazirani, 2001; Glover and Greenberg, 1989; Zlochin et al., 2004).

2.4.2 Genetic Programming

Genetic algorithms evolve a population of candidate solutions by selection, crossover, and mutation. Genetic programming (*Genetic Programming* (GP)) is a specialised form of genetic algorithm that evolves computer programmes, typically represented as trees whose nodes are functions or terminals (Koza, 1992). An operator of selection uses fitness to choose parents; crossover mixes elements of different parent solutions to produce offspring; mutation introduces random variation; and a survival or replacement operator forms the next generation until a stopping criterion is met.

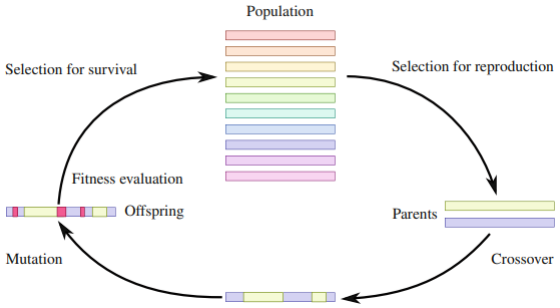


Figure 2.3: Generational loop in an evolutionary algorithm. From a population of solutions, individuals are selected to reproduce by crossover and mutation; offspring are evaluated and incorporated; and a survival operator restores the population size. Source: adapted from standard evolutionary-algorithm schematics (see also Koza (1992)).

GP is particularly effective for symbolic regression, automated algorithm discovery, and optimisation tasks (Koza, 1992; Banzhaf et al., 1998). Population size and the number of generations are crucial parameters that influence exploration of the search space and convergence speed.

Grammatical Evolution

Grammatical Evolution (GE) was introduced by O'Neill and Ryan as a way to apply evolutionary computing to problems defined by a formal grammar (O'Neill and Conor Ryan, 2001; O'Neill and C. Ryan, 2003). A formal grammar is a set of rules that define a language's syntax and allow generation of valid strings or structures. In GE, the genotype is an integer vector rather than a tree; the grammar maps those vectors to valid phenotypes. This representation supports flexible adaptation to different grammar structures.

The process of Grammatical Evolution involves the following steps:

1. Define a BNF representation of the grammar;
2. Randomly generate genotypes as integer vectors;
3. Evaluate fitness by mapping genotypes to phenotypes and testing them against the fitness function;
4. Use genetic operators such as crossover and mutation to evolve the population of genotypes;
5. Repeat steps 3 and 4 until a satisfactory solution is found or a stopping criterion is met.

2.4.3 Backus-Naur Form

A BNF grammar describes the syntax of a language as a tuple of four sets:

- T: the terminal set;
- N: the non-terminal set;
- P: the set of production rules;

- S: the start symbol (a member of N).

For example, consider a simple grammar for mathematical expressions with addition, subtraction, multiplication, and division, using variables x and y:

$$T = \{+, -, /, *, x, y\}, \qquad N = \{\langle e\rangle, \langle o\rangle, \langle v\rangle\}.$$

The non-terminal $\langle e\rangle$ denotes an expression, $\langle o\rangle$ an operator, and $\langle v\rangle$ a variable. The production rules in P are:

$$\langle e\rangle ::= \langle e\rangle\langle o\rangle\langle e\rangle \mid \langle v\rangle \tag{2.1}$$
$$\langle o\rangle ::= + \mid - \mid / \mid * \tag{2.2}$$
$$\langle v\rangle ::= x \mid y \tag{2.3}$$

The pipe operator | denotes alternatives. The start symbol is $S = \langle e\rangle$.

Genotypes may be represented as non-negative integer arrays. For example, consider

$$C = [4, 15, 75, 8, 41, 12].$$

Mapping begins from the start symbol $\langle e\rangle$. The general rule is

$$\text{selected_option} = \text{gene_value} \bmod r,$$

where gene_value is the current integer from the genotype, r is the number of production choices available for the current non-terminal, and selected_option is the (zero-based) index of the chosen production.

The first gene is 4 and $\langle e\rangle$ has two options, so $4 \bmod 2 = 0$ selects $\langle e\rangle\langle o\rangle\langle e\rangle$. Continuing left-to-right:

1. With gene 15, $15 \bmod 2 = 1$ replaces the first $\langle e\rangle$ with $\langle v\rangle$;
2. With gene 75, $75 \bmod 2 = 1$ replaces $\langle v\rangle$ with y;
3. With gene 8, $8 \bmod 4 = 0$ replaces $\langle o\rangle$ with $+$;
4. With gene 41, $41 \bmod 2 = 1$ replaces the remaining $\langle e\rangle$ with $\langle v\rangle$;
5. With gene 12, $12 \bmod 2 = 0$ replaces $\langle v\rangle$ with x.

The final expression is $y + x$.

2.4.4 Initialization Phase

Initialisation can be performed on the genome or on the derivation tree. To control tree depth, this work uses Position-Independent Grow (*Position-Independent Grow* (PIG)), which generates individuals that tend to fill the allowed depth range: neither systematically too deep nor too shallow. PIG expands non-terminals in random order rather than always expanding leftmost first, and ensures that at least one branch reaches the maximum depth. This reduces duplicate structures and increases diversity in the initial population.

2.4.5 Selection Mechanism

The selection mechanism chooses individuals from the population to act as parents for the next generation while retaining some randomness to preserve diversity. This work adopts tournament selection: a number of individuals are drawn at random, the best individual in each tournament is retained, and the process is repeated until the required number of parents has been selected.

2.4.6 Variation Operators

The variation operators explore the search space by modifying genetic material. The two primary operators adopted in this work are crossover and mutation.

The crossover operator mixes parent genotypes to obtain offspring genotypes. Variable one-point crossover, which was chosen for this work, swaps the tails of two parents after cut points selected independently in each parent; because the cut positions need not coincide, offspring genome lengths may differ from those of the parents, enabling combination of beneficial traits of different lengths.

Mutation is applied to every child after crossover, using a codon-based integer-flip method. Here a codon is one element of the genotype vector: an integer flip replaces that codon with another random integer in the allowed codon range. Each codon is mutated independently with probability

$$P_{\text{mutation}} = \frac{1}{\text{genome_length}}. \tag{2.4}$$

In standard PonyGE2 practice this probability is applied over the genome (or over used codons, depending on configuration); the operator itself does not "know" which unused trailing genes will later be consumed by wrapping or unused material.

2.4.7 Replacement Strategy

The replacement strategy determines how new individuals enter the population. This work uses generational replacement with elitism: the best individuals from the previous generation are retained, while the remaining positions are filled by newly generated offspring. Elitism helps preserve high-quality solutions while the rest of the population continues to explore.

2.5 Metrics

2.5.1 Precision and Recall

In anomaly detection, precision and recall provide complementary perspectives on model performance, especially under class imbalance (e.g., rare fraud cases among many legitimate events).

Precision

Precision quantifies a model's ability to minimise false-positive classifications:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \tag{2.5}$$

where TP denotes true positives and FP false positives.

Recall

Recall evaluates how completely positive cases are identified:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \tag{2.6}$$

where FN denotes false negatives.

2.5.2 Trade-off and Applications

Raising the decision threshold typically increases precision and decreases recall, and vice versa; the preferred balance is domain-specific. High recall is paramount in medical diagnostics, where missing a true positive can be severe. High precision is crucial in spam filtering, where false positives waste user attention.

F1 Score

The F1 score is the harmonic mean of precision and recall:

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{2.7}$$

It provides a single score when precision and recall should be balanced, but can be misleading under severe class imbalance, where AUC or precision–recall curves may be more appropriate.

2.5.3 Area Under the ROC Curve (AUC)

The *Receiver Operating Characteristic* (ROC) curve plots the true-positive rate against the false-positive rate across classification thresholds (Hanley and McNeil, 1982). The Area Under the ROC Curve (AUC) is a common performance metric for binary classifiers under class imbalance (Fawcett, 2006). It measures the classifier's capacity to discriminate between classes.

Interpretation and Calculation

AUC values range from 0 to 1, where:

- 0.5 indicates random guessing;
- 1 represents perfect separability;
- values below 0.5 suggest inverted ranking.

The Wilcoxon–Mann–Whitney statistic provides a non-parametric AUC estimate (Hanley and McNeil, 1982):

$$\text{AUC} = \frac{\sum_{i=1}^{n_+} \sum_{j=1}^{n_-} \mathbf{1}(s_i^+ > s_j^-)}{n_+ \cdot n_-}, \tag{2.8}$$

where n_+ and n_- denote the numbers of positive and negative instances and s denotes classifier scores.

Advantages and Limitations

AUC is robust to class imbalance and does not require a fixed threshold (Bradley, 1997). However, it does not capture cost-sensitive decision contexts and can be misleading when ROC curves cross (Lobo et al., 2008).

Chapter 3

Literature Review

3.1 Anomaly Detection in Telecommunications Networks

Telecommunication anomaly detection is difficult because labels are scarce and data volumes are large. Often the harder operational question is not only how to score a series, but which subsection of the network view to monitor.

Gonzalez et al. (2024) apply a Dilated Convolutional Variational Autoencoder (DC-*Variational Autoencoder* (VAE)) to multivariate mobile-network counters on a fixed set of 12 features. Of about 730,000 samples, only about 9,000 were expert-labelled. Using range-based F1 (range-F1), which scores overlap between predicted and labelled anomaly intervals rather than isolated timestamps, they report a mean range-F1 of about 52% on that fixed, fully aggregated multivariate view. The result shows that even a modern multivariate deep detector remains only modest under sparse labels when the monitored view is fixed in advance.

Burgueno et al. (2020) analysed a network with about 25,000 mobile base-station coverage areas. For each cell they could measure 240 features, but the study focuses on 15 selected ones. From this perspective, manual selection is a bottleneck because the remaining unmonitored features may still contain relevant information.

Call Detail Record (CDR) and mobile activity records are widely used in telecom monitoring. Sultan et al. (2018) focused on traffic prediction rather than anomaly detection, and no detection metrics are reported; the paper is therefore of limited use as a detection baseline.

Lemaire et al. (2024) study early detection of critical urban events from Orange France mobile Network Signalling Data in Paris: cell-level, one-minute counts of network services (calls, SMS, and related signalling), with hand-designed service aggregations and fusion. Against a curated set of documented urban events, their best precision is 28.63%, with event recall of about 36% at the corresponding operating point, indicating many false alarms even when the signal is useful. The aggregations are categorical service views at cell level, not searched hierarchical structures.

Existing work therefore often assumes a fixed aggregation or a manually chosen feature subset, and may miss anomalies that would appear only under a different grouping. Automated search is useful when the best aggregation is not obvious in advance. The next sections examine grouping, hierarchy, scoring, and combinatorial search.

3.2 Anomaly Detection in Grouped Time Series

Anomaly detection in grouped time series treats the detector input as a chosen representation, not the whole network: series are aggregated by related features or entities before scoring. Detection quality therefore depends on that representation, not only on the detector.

Across the reviewed studies, datasets, labels, and metrics differ, so reported scores are not directly comparable.

Mefleh et al. (2025) used unsupervised Mahalanobis distance on telecommunications *Key Performance Indicator* (KPI)s. They report AUC rising from 0.77 on raw KPI features to 0.86 when KPI denominators (e.g., attempt counts that normalise ratios) are included, then to 0.93 with adjusted ratios, and finally to 0.99 with learned adjustments. Denominator information is therefore a representation choice related to aggregation and normalisation, not a change of detector family. The result shows that detection quality changes strongly with feature transformation and aggregation.

Hua et al. (2022) introduce GenAD, a generative model for multivariate time series that exploits shared operational behaviour across China Mobile base stations. It was pre-trained on 3,000 unlabeled base stations and then used to detect anomalies on other stations, reporting F1 of 0.953 (precision 0.910). Their evaluation used point-adjusted segment scoring, treating detection over anomaly intervals rather than isolated timestamps.

Oguz and Kalaycioglu (2022) simulate a multi-tier cellular network and classify the state of each femtocell access point from aggregated user-equipment measurements, comparing *Long Short-Term Memory* (LSTM) and 1D-*Convolutional Neural Network* (CNN) detectors. Performance changes with the region of interest (*Region of Interest* (ROI)) radius, signal shadowing, and aggregation methods. At a 30 m ROI with 2 dB shadowing, the 1D CNN achieved 80.1% accuracy, but accuracy falls as ROI size and shadowing increase, showing that including more observations is not necessarily beneficial when added signals are noisier or less relevant.

Overall, the reviewed studies show that grouping and representation strongly affect anomaly detection performance. Aggregations are typically selected manually before modelling; none systematically search a grammar-defined space of valid hierarchical aggregations of telecommunication data. That search is the focus of this dissertation.

3.3 Anomaly Detection in Hierarchical Time Series

In telecom operations, an alert that "traffic is abnormal" is not enough: the operator must locate whether the effect is network-wide, regional, cell- or switch-specific, or confined to one KPI category. Hierarchy supplies that location structure; a flat grouping does not. A fixed top-down hierarchy can help localisation even when it is not the best aggregation for detection itself.

Hong et al. (2012) detect and localise anomalies in hierarchical *Internet Service Provider* (ISP) call-detail data using customer-care logs and signalling records across predefined hierarchies. They report finding over 94% of reference anomalies and 99.7% anomaly identification accuracy under a fixed additive hierarchy with Holt–Winters spike forecasting. The work demonstrates hierarchical localisation rather than search over alternative partial aggregations.

Wolff et al. (2022) treat anomaly localisation as sequential search over a known tree of processes: their hierarchical dynamic search first samples aggregate internal nodes and then moves toward the branch most likely to contain an anomaly. This shows that hierarchy can be an active search space, not only a reporting structure after detection. However, the tree topology is assumed known; search stays inside it. This differs from the present approach, which uses GE to generate candidate aggregations before applying the anomaly scorer.

Huang et al. (2026) propose HYDRA, a multi-level hierarchy-driven anomaly detector that builds multiple hierarchy-based views of each series and combines detection signals across levels. It ranked first among 40 algorithms across 40 TSB-AD datasets. Unlike the telecom hierarchy papers above, HYDRA addresses general benchmark time series with predefined multi-level representations rather than searching partial aggregations over operational categorical dimensions.

3.4 Matrix Profile and Time-Series Discord Detection

Chapter 2 introduced anomaly types and the need for a label-light scorer. Here the focus is why Matrix Profile (*Matrix Profile* (MP)) fits the evolutionary search setting: GE generates many candidate aggregations, and each resulting series needs a fast, exact, largely unsupervised discord score.

A time-series discord is the subsequence most distant from its nearest non-self match (Keogh et al., 2005). On the Matrix Profile, the discord is the position with the maximum nearest-neighbour distance (Yeh et al., 2017). Distance is typically z-normalised Euclidean distance between subsequences of fixed length. Discords capture short unusual spikes well, but may poorly capture sustained level shifts, gradual ramps, or purely relational multivariate contextual faults.

Computational efficiency matters when many candidates must be scored: a refinery time-series self-join is reduced from an estimate of 153.8 days by brute force to 1.2 hours with STAMP/STOMP (Yeh et al., 2017), and Zhu et al. (2016) showed GPU-STOMP processing 100 million subsequences in about 12 days versus an estimate of 1585 years for brute force. STUMPY (Law, 2019) provides a practical Python implementation. Although Matrix Profile can score a candidate series, it does not choose which aggregation to score; that selection problem motivates combinatorial optimisation.

3.5 Combinatorial Optimisation for Selecting Time Series

Little published work searches hierarchical aggregation structures for telecom anomaly detection. Related combinatorial work on feature or subset selection is reviewed here because, from an optimisation viewpoint, both settings enumerate large candidate spaces and score each candidate with a fitness function. The spaces are not identical: feature selection chooses columns, whereas this dissertation searches grammar-constrained hierarchies.

Feature/subset selection is combinatorial: the candidate space is of size 2^n, so full enumeration fails for large n (Xue et al., 2016). Evolutionary algorithms are widely used

to search such spaces (Xue et al., 2016).

Linares-Barrera et al. (2024) use a genetic algorithm (SEISEN) to select lagged forecasting features on an electricity dataset, reducing the feature set from 7,704 to 3,813 and improving *Mean Absolute Error* (MAE) from 207.193 to 194.400.

Contreras et al. (2024) study financial direction prediction with a genetic algorithm whose fitness is AUC, reporting modest test AUC values of about 0.58–0.62 for candidate feature subsets.

Closer to anomaly detection, He et al. (2021) combine subspace search (greedy or evolutionary $(\mu+\lambda)$ heuristics) with discord mining on high-dimensional financial transaction time series, showing that searching subspaces beats scoring all dimensions or a single best dimension on retrieval metrics. Li et al. (2022) couple a variational autoencoder scorer with a genetic algorithm that searches abnormal subspaces, improving outlier AUC over a plain VAE on several high-dimensional benchmarks.

These studies support pairing a fast scorer with evolutionary search over structured candidates. None integrate grammar-constrained hierarchy search, Matrix Profile scoring, telecommunications operational data, and AUC-based evaluation in one pipeline—the gap addressed by this dissertation.

Chapter 4

Materials and Methodology

4.1 Problem Formulation

This research develops a hierarchical anomaly detection framework for mobile network data to improve service reliability. The study focuses on grouped time series from a network switch (July–Oct 2022), analyzing 4.2M hourly records of call entries/exits. Two core metrics are tracked: #Events (call frequency) and Event Duration (total call time), aggregated across six dimensions like traffic type (ON/OFF-NET) and subscriber category (Roaming/Postpaid).

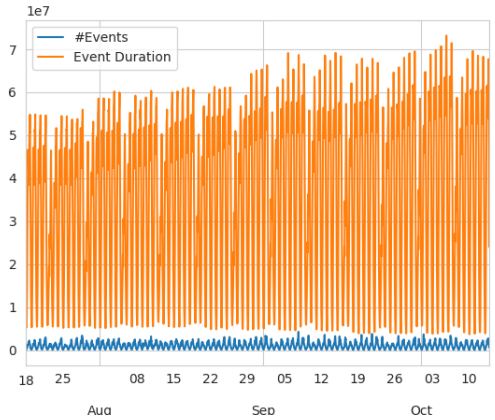


Figure 4.1: Hourly network traffic patterns showing (A) Total #Events (left axis) and Event Duration in hours (right axis).

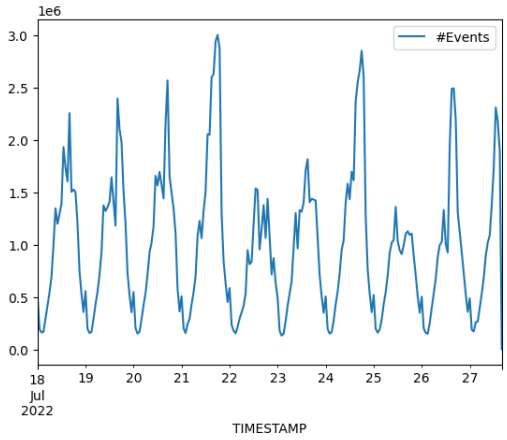


Figure 4.2: Hourly call event frequency.

Table 4.1: Cardinality and Dominant Categories in Vodafone Data

Variable	Unique Values	Top Category	Frequency
Call Type	1,555	VOICE	849 (54.6%)
Event Type	1,555	ORIGINATED	716 (46.0%)
Subscriber Type	1,555	Roamer In	1,486 (95.6%)
Traffic Type	1,555	INT	1,486 (95.6%)
Service Type	1,555	NOT-ROAMING	1,544 (99.3%)
Partner Description	1,555	P_0001	65 (4.2%)

4.1.1 Key Observations

The categorical variable analysis reveals three distinct distribution patterns:

- **Highly Skewed Variables**:
  - Subscriber Type: 95.6% Roamer In
  - Traffic Type: 95.6% INT
  - Service Type: 99.3% NOT-ROAMING
- **Balanced Variables**:
  - Call Type: 54.6% VOICE vs 45.4% SMS
  - Event Type: 46.0% ORIGINATED vs 54.0% other
- **High-Cardinality Variable**:
  - Partner Description: 1,555 unique values (long-tail distribution)
  - Top partner P_0001 covers only 4.2% of data

4.1.2 Analytical Implications

$$\text{Information Gain} = -\sum_{i=1}^{n} p_i \log_2 p_i \tag{4.1}$$

Where p_i is the proportion of each category. Calculated entropy values:

Table 4.2: Information Content by Variable

Variable	Entropy (bits)
Call Type	0.996
Event Type	1.000
Subscriber Type	0.286
Traffic Type	0.286
Service Type	0.048
Partner Description	7.602

The high entropy of Partner Description (7.6 bits) suggests it requires special handling during hierarchical aggregation to avoid overfitting. Conversely, near-zero entropy of Service Type indicates limited predictive value for anomaly detection.

4.2 Adopted Methodology

The original dataset contains approximately 1M records with six features after filtering and removing empty timestamps. Aggregation is performed over combinations of the six categorical features and their distinct values, using #Events as the target variable. This yields 1,555 non-empty unique feature combinations out of 94,080 theoretically available combinations. The mean number of timestamps among non-empty combinations is 569, which means that the dataset is highly sparse. To create anomalies, 5% of the timestamps are randomly selected and modified to include synthetic anomalies. Finally, the dataset is split into training (80%) and testing (20%) sets.

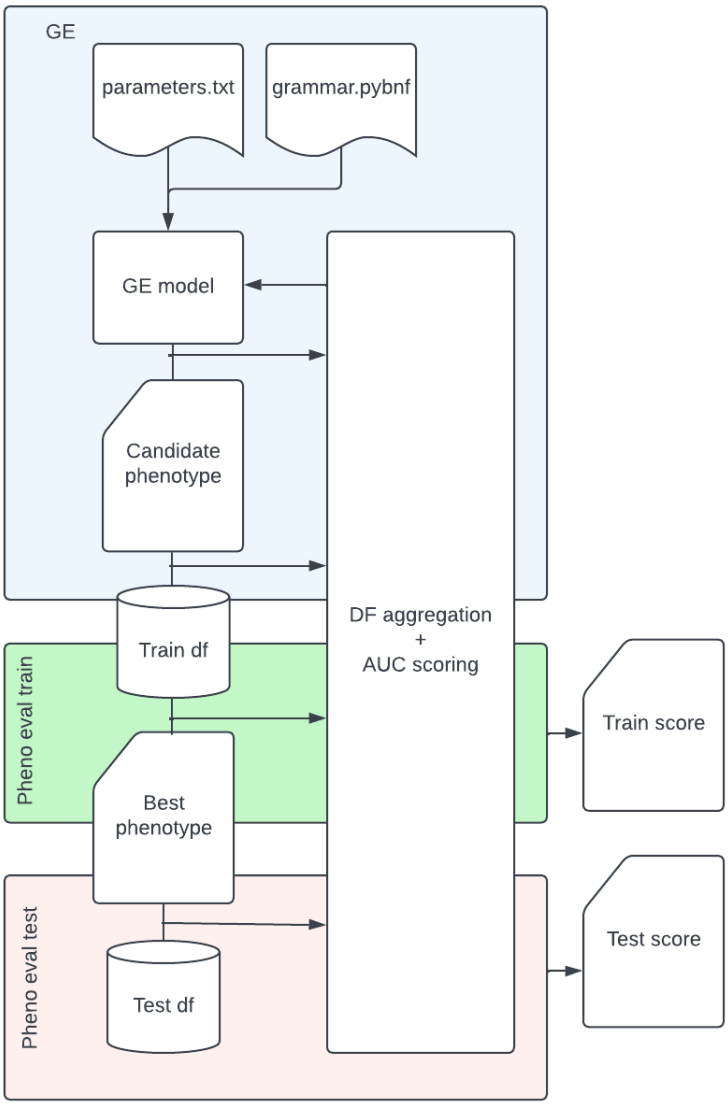


Figure 4.3: Hierarchical anomaly detection pipeline combining data aggregation, synthetic anomaly injection, and genetic model optimization.

Workflow Stages:

1. **Data Preparation**
   - Input: 1M records (6 features)
   - Filter: Remove empty timestamps
   - Aggregate: Random feature combinations
2. **Anomaly Injection**
   - Insert 5% synthetic timestamp anomalies
   - Quality Check: "Verify anomaly detection effectiveness"
3. **Data Partitioning**

- Split into training (80%) and testing (20%) sets

4. **Model Development**
    - Grammar-based evolution
    - Inputs: Rule templates + Parameters
5. **Evaluation & Refinement**
    - Output: Best-performing detection rules
    - Metrics: AUC-ROC scores
    - Iterative parameter tuning

Key Processes:

- Hierarchy parsing → Condition filtering → Temporal aggregation
- Matrix Profile analysis → Weighted AUC calculation
- Data gap filling → Event count normalization

Symbol Legend:

- $\rightarrow$: Process flow direction
- **AUC** : Area Under Curve
- **DF** : dataframe

4.3 Model Development and Evaluation

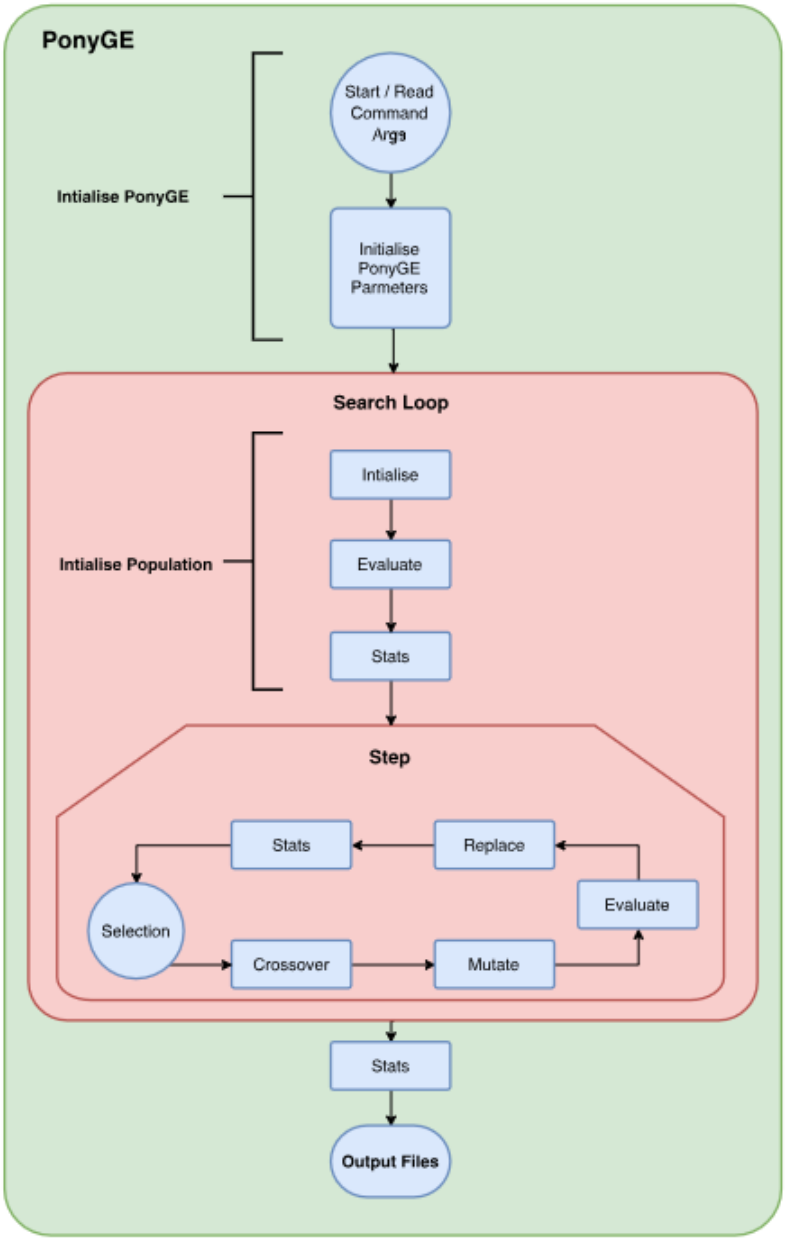


Figure 4.4: PonyGE2 control flow diagram (Fenton et al., 2017).

4.3.1 Genome Structure

The genome structure is defined by codons: each codon is an integer in the range $[0, 99{,}999]$, corresponding to a codon size of 100,000. Each codon value determines the selection of a grammar rule during programme construction. Defining a maximum genome length is crucial for preventing unbounded growth of solutions; it represents the maximum "DNA length" of an individual.

4.4 Anomaly Injection

The anomaly injection process generates synthetic anomalies to simulate labelled evaluation scenarios. Since the operational dataset does not contain reliable anomaly labels, injection is essential for evaluating detection performance. The procedure defines anomaly parameters, selects feature combinations, and modifies records to create anomalies while preserving the rest of the series.

For anomaly generation, the window size is 168 hours (one week) and the threshold is $\mu \pm 3\sigma$ relative to the historical baseline. Each anomaly is formed by selecting categorical features independently with probability below 50% each, sampling a value within each selected feature to obtain a combination (e.g., Service=ROAMING, Traffic Type=INT), and then modifying the matching records within a 168-hour window. For each selected group, the #Events value is set outside the normal range ($\mu \pm 3\sigma$), and an anomaly flag is assigned. This distributes contextually realistic anomalies across diverse subsets.

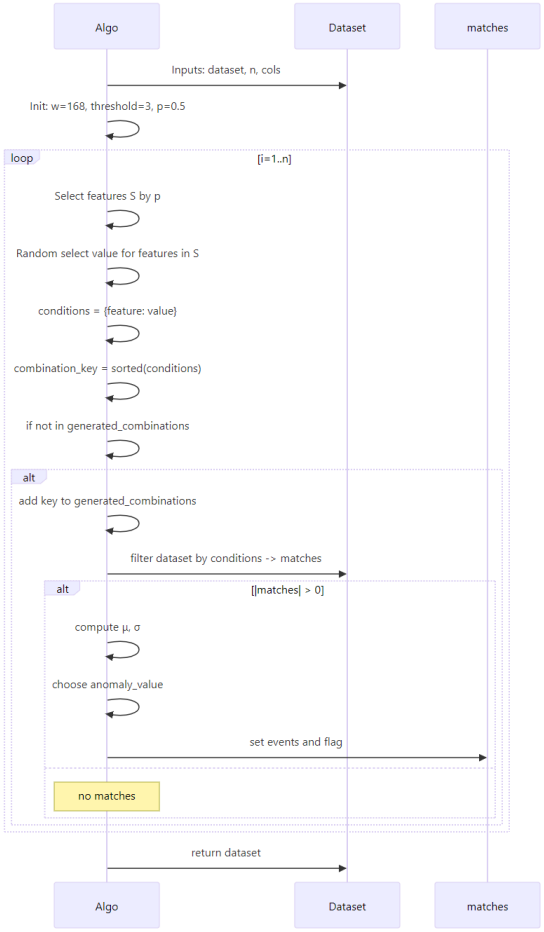


Figure 4.5: Anomaly injection cycle.

4.5 Hierarchical Data Structures

There is no unique way to aggregate the data, and admissible aggregations are limited by the filters applied. Filters may be empty or not; using the list of non-empty feature-value combinations limits the aggregations that can be formed and helps avoid empty

timestamps.

The data is represented as a list of dictionaries, where each dictionary contains the values of the features and the target variable (#Events).

4.6 Genetic Programming Implementation

The genetic programming implementation is based on the PonyGE2 framework, which has the following key components:

Table 4.3: Key Configuration Parameters

Parameter	Value
CODON_SIZE	100000
CROSSOVER	variable_onepoint
CROSSOVER_PROBABILITY	0.75
GENERATIONS	20-100
MAX_GENOME_LENGTH	500
INITIALISATION	PI_grow
MAX_INIT_TREE_DEPTH	5
MAX_TREE_DEPTH	10
MUTATION	int_flip_per_codon
POPULATION_SIZE	10-50
FITNESS_FUNCTION	hierarchy_auc
REPLACEMENT	generational
SELECTION	tournament
TOURNAMENT_SIZE	2

Chapter 5

Results and Discussion

5.1 Performance Analysis

5.2 Time Series Visualization

5.3 Genetic Programming Analysis

Chapter 6

Conclusions and Future Works

6.1 Conclusions

6.2 Future Work

Bibliography

Athanasopoulos, George et al. (2017). "Forecasting with temporal hierarchies". In: *European Journal of Operational Research* 262.1, pp. 60–74.

Banzhaf, W. et al. (1998). *Genetic Programming: An Introduction*. Morgan Kaufmann.

Box, George EP et al. (2015). *Time series analysis: Forecasting and control*. 5th. John Wiley & Sons.

Bradley, Andrew P (1997). "The use of the area under the ROC curve in the evaluation of machine learning algorithms". In: *Pattern recognition* 30.7, pp. 1145–1159.

Burgueno, Jesus et al. (2020). "Online Anomaly Detection System for Mobile Networks". In: *Sensors* 20.24, p. 7232. DOI: 10.3390/s20247232.

Chandola, Varun, Arindam Banerjee, and Vipin Kumar (2009). "Anomaly detection: A survey". In: *ACM computing surveys (CSUR)* 41.3, pp. 1–58.

Contreras, Rodrigo Caetano et al. (2024). "Genetic Algorithm for Feature Selection Applied to Financial Time Series Monotonicity Prediction: Experimental Cases in Cryptocurrencies and Brazilian Assets". In: *Entropy* 26.3, p. 177. DOI: 10.3390/e26030177.

Fawcett, Tom (2006). "An introduction to ROC analysis". In: *Pattern recognition letters* 27.8, pp. 861–874.

Fenton, Michael et al. (2017). "Ponyge2: Grammatical evolution in python". In: *Proceedings of the genetic and evolutionary computation conference companion*, pp. 1194–1201.

Garey, M. R. and D. S. Johnson (1979). *Computers and intractability: A guide to the theory of NP-completeness*. W.H. Freeman and Company.

Glover, Fred and Harvey J Greenberg (1989). "New approaches for heuristic search: A bilateral linkage with artificial intelligence". In: *European Journal of Operational Research* 39.2, pp. 119–130.

Gonzalez, Guillermo Garcia et al. (2024). "One Model to Find Them All: Deep Learning for Multivariate Time-Series Anomaly Detection in Mobile Network Data". In: *IEEE Transactions on Network and Service Management* 21.2, pp. 1601–1616. DOI: 10.1109/TNSM.2023.3340146.

Hanley, James A and Barbara J McNeil (1982). "The meaning and use of the area under a receiver operating characteristic (ROC) curve." In: *Radiology* 143.1, pp. 29–36.

Hawkins, Douglas M (1980). *Identification of outliers*. Vol. 11. Springer.

He, Jieming et al. (2021). "Mining Anomalies in Subspaces of High-Dimensional Time Series for Financial Transactional Data". In: *Machine Learning and Knowledge Discovery in Databases. Applied Data Science Track*, pp. 19–36. DOI: 10.1007/978-3-030-86514-6_2.

Hong, Chi-Yao et al. (2012). "Tiresias: Online Anomaly Detection for Hierarchical Operational Network Data". In: *2012 IEEE 32nd International Conference on Distributed Computing Systems*, pp. 173–182. DOI: 10.1109/ICDCS.2012.30.

Hua, X. et al. (2022). "GenAD: General Unsupervised Anomaly Detection Using Multivariate Time Series for Large-Scale Wireless Base Stations". In: *Electronics Letters* 59.1. DOI: 10.1049/ell2.12683.

Huang, Ming et al. (2026). "HYDRA: A Multi-Level Hierarchy-Driven Approach for Robust Anomaly Detection in Time Series". In: *Proceedings of the ACM on Management of Data* 4.3, pp. 1–30. DOI: 10.1145/3802074.

Hyndman, R. J. and G. Athanasopoulos (2018). *Forecasting: Principles and Practice*. 2nd ed. Retrieved from https://otexts.com/fpp2/. OTexts.

— (2021). "Chapter 11 Forecasting hierarchical and grouped time series". In: *Forecasting: Principles and Practice*.

Keogh, Eamonn, Jessica Lin, and Ada Fu (2005). "HOT SAX: Efficiently Finding the Most Unusual Time Series Subsequence". In: *Fifth IEEE International Conference on Data Mining*, pp. 226–233. DOI: 10.1109/ICDM.2005.79.

Koza, J. R. (1992). *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press.

Law, Sean (2019). "STUMPY: A Powerful and Scalable Python Library for Time Series Data Mining". In: *Journal of Open Source Software* 4.39, p. 1504. DOI: 10.21105/joss.01504.

Lemaire, Pierre et al. (2024). "Early Detection of Critical Urban Events Using Mobile Phone Network Data". In: *PLOS ONE* 19.8, e0309093. DOI: 10.1371/journal.pone.0309093.

Li, Jian et al. (2022). "An Auto-Encoder with Genetic Algorithm for High Dimensional Data: Towards Accurate and Interpretable Outlier Detection". In: *Algorithms* 15.11, p. 429. DOI: 10.3390/a15110429.

Linares-Barrera, Maria Luisa et al. (2024). "Evolutionary Feature Selection for Time-Series Forecasting". In: *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*, pp. 395–399. DOI: 10.1145/3605098.3636191.

Lobo, Jorge M, Alberto Jiménez-Valverde, and Raimundo Real (2008). "AUC: a misleading measure of the performance of predictive distribution models". In: *Global ecology and Biogeography* 17.2, pp. 145–151.

Mefleh, A. et al. (2025). "Beyond the Norm: Unsupervised Anomaly Detection in Telecommunications with Mahalanobis Distance". In: *Computers* 14.12, p. 561. DOI: 10.3390/computers14120561.

O'Neill, Michael and Conor Ryan (2001). "Grammatical Evolution". In: *IEEE Transactions on Evolutionary Computation* 5.4, pp. 349–358. DOI: 10.1109/4235.942529.

O'Neill, M. and C. Ryan (2003). *Grammatical Evolution: Evolutionary Automatic Programming in an Arbitrary Language*. Springer.

Oguz, H. T. and A. Kalaycioglu (2022). "Anomaly Detection in Multi-Tiered Cellular Networks Using LSTM and 1D CNN". In: *EURASIP Journal on Wireless Communications and Networking* 2022.1. DOI: 10.1186/s13638-022-02183-7.

Papadimitriou, C. H. and K. Steiglitz (1982). *Combinatorial optimization: Algorithms and complexity*. Prentice Hall.

Papadimitriou, Christos H and Kenneth Steiglitz (1998). *Combinatorial optimization: algorithms and complexity*. Courier Corporation.

Ruff, Lukas et al. (2021). "A unifying review of deep and shallow anomaly detection". In: *Proceedings of the IEEE* 109.5, pp. 756–795.

Sultan, Kashif, Hazrat Ali, and Zhang Zhang (2018). "Call Detail Records Driven Anomaly Detection and Traffic Prediction in Mobile Cellular Networks". In: *IEEE Access* 6, pp. 41728–41737. DOI: 10.1109/ACCESS.2018.2859756.

Vazirani, Vijay V (2001). *Approximation algorithms*. Vol. 1. Springer.

Wickramasuriya, Shanika L, George Athanasopoulos, Rob J Hyndman, et al. (2015). "Forecasting hierarchical and grouped time series through trace minimization". In: *Department of Econometrics and Business Statistics, Monash University* 105, pp. 6–1.

Wickramasuriya, Shanika L, George Athanasopoulos, and Rob J Hyndman (2019). "Optimal forecast reconciliation for hierarchical and grouped time series through trace minimization". In: *Journal of the American Statistical Association* 114.526, pp. 804–819.

Wolff, Bar et al. (2022). "Composite Anomaly Detection via Hierarchical Dynamic Search". In: *2022 IEEE International Symposium on Information Theory*, pp. 2421–2426. DOI: 10.1109/ISIT50566.2022.9834631.

Xue, Bing et al. (2016). "A Survey on Evolutionary Computation Approaches to Feature Selection". In: *IEEE Transactions on Evolutionary Computation* 20.4, pp. 606–626. DOI: 10.1109/TEVC.2015.2504420.

Yeh, Chin-Chia Michael et al. (2017). "Time Series Joins, Motifs, Discords and Shapelets: A Unifying View That Exploits the Matrix Profile". In: *Data Mining and Knowledge Discovery* 32.1, pp. 83–123. DOI: 10.1007/s10618-017-0519-9.

Zhu, Yan et al. (2016). "Matrix Profile II: Exploiting a Novel Algorithm and GPUs to Break the One Hundred Million Barrier for Time Series Motifs and Joins". In: *2016 IEEE 16th International Conference on Data Mining*, pp. 739–748. DOI: 10.1109/ICDM.2016.0085.

Zlochin, Mark et al. (2004). "Model-based search for combinatorial optimization: A critical survey". In: *Annals of Operations Research* 131, pp. 373–395.